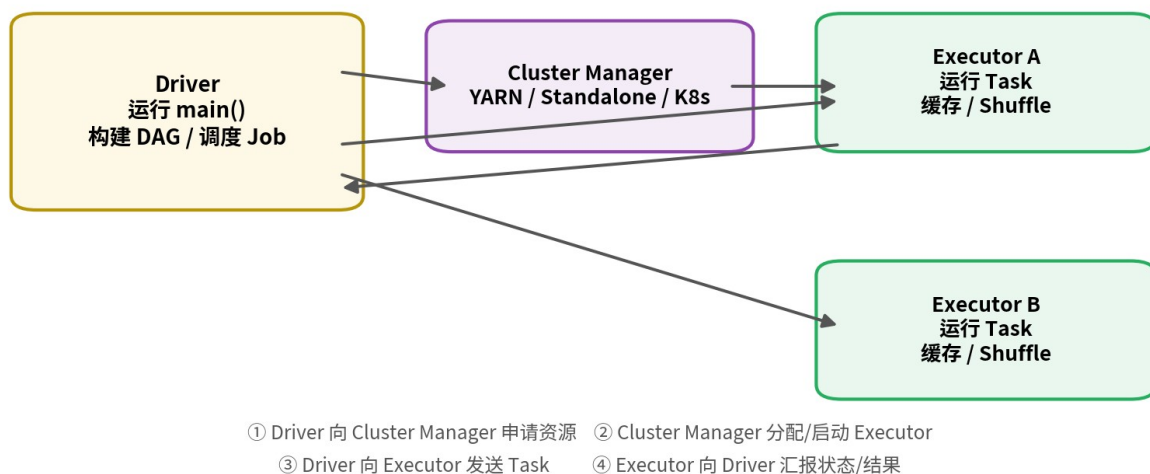


大数据处理技术：Spark 原理入门

第十二讲：一段 PySpark 代码到底是怎样跑起来的

SparkSession • SparkContext • Driver • Executor • Job • Stage • Task

Spark Application : Driver、Cluster Manager 与 Executor



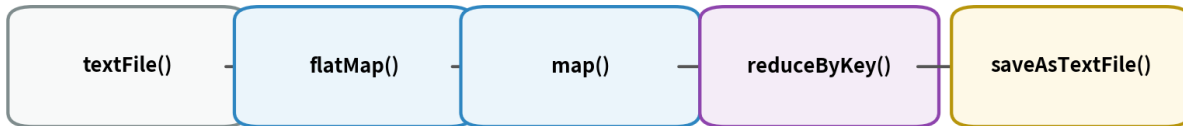
一句话：Driver 负责“想清楚怎么干”，Executor 负责“真正把各个分区算掉”。

本讲目标：拿一段真正的 PySpark WordCount，从代码一直追踪到分区、Stage、Task 和 Executor。

一、先建立总图：Spark 程序不是“Python 一行一行直接算”

初学者最容易形成的错觉是：看到 Python 代码从上往下写，就以为 Spark 也从上往下一行行把全部数据算完。实际上，大量 Transformation 只是向 Driver 描述“数据应该怎样变化”。直到 Action 出现，Spark 才把整条依赖关系转换成可执行的 Job。

一段 PySpark 代码如何变成真正的分布式执行



前四步：主要记录依赖关系（Transformation）

Action：触发真正执行

Action 出现后：Driver 创建 Job，并把依赖图交给调度体系



你写“做什么”；Spark 决定“按什么 Stage、多少 Task、在哪些 Executor 上执行”。

最核心的一句话：Driver 先构建“计算计划”；Action 触发 Job；DAG Scheduler 按 Shuffle 等边界切 Stage；每个 Stage 再按照 Partition 生成 Task，最后由 Executor 真正执行。

二、先看一份完整 PySpark WordCount

完整示例（PySpark RDD API）

```

from pyspark.sql import SparkSession

# 1. 创建 SparkSession: 现代 Spark 应用的统一入口
spark = SparkSession.builder \
    .appName("WordCount") \
    .getOrCreate()

# 2. 取得底层 SparkContext (RDD API 会用到它)
sc = spark.sparkContext

# 3. 从 HDFS/文件系统读取文本, 得到 RDD[String]
lines = sc.textFile("hdfs:///input/words.txt")

# 4. 每行拆成多个单词: 一条记录 -> 多条记录
words = lines.flatMap(lambda line: line.split())
  
```

Spark 执行原理入门：从 PySpark 代码到 Driver / Executor / Job / Stage / Task

```
# 5. 变成 (word, 1)
pairs = words.map(lambda word: (word, 1))

# 6. 相同 word 跨分区聚合；这里通常会发生 Shuffle
counts = pairs.reduceByKey(lambda a, b: a + b)

# 7. Action: 真正触发计算，并把结果写出去
counts.saveAsTextFile("hdfs:///output/wordcount")

spark.stop()
```

这份程序很短，但背后包含 Driver、Executor、Partition、Shuffle、Job、Stage、Task 等一整套分布式机制。下面逐层拆开。

三、SparkSession 和 SparkContext 到底是什么？

对象	可以怎样理解	主要职责
SparkSession	现代 Spark 的“总入口”	统一访问 DataFrame/SQL，同时可以取得 SparkContext
SparkContext	更底层的“应用与集群连接/调度上下文”	创建 RDD、与集群资源和任务调度体系协作

在现代 PySpark 程序里，一般先创建 SparkSession，然后通过 spark.sparkContext 获得 SparkContext。不要把它们理解成两套互相竞争的 Spark；SparkSession 是更高层的统一入口。

迁移理解：如果把 Hadoop Job 的 Driver 配置代码看成“任务入口”，SparkSession/Driver 则更进一步：它不仅定义任务，还在整个 Spark Application 生命周期里持续存在并负责调度。

四、Driver: Spark Application 的“指挥部”

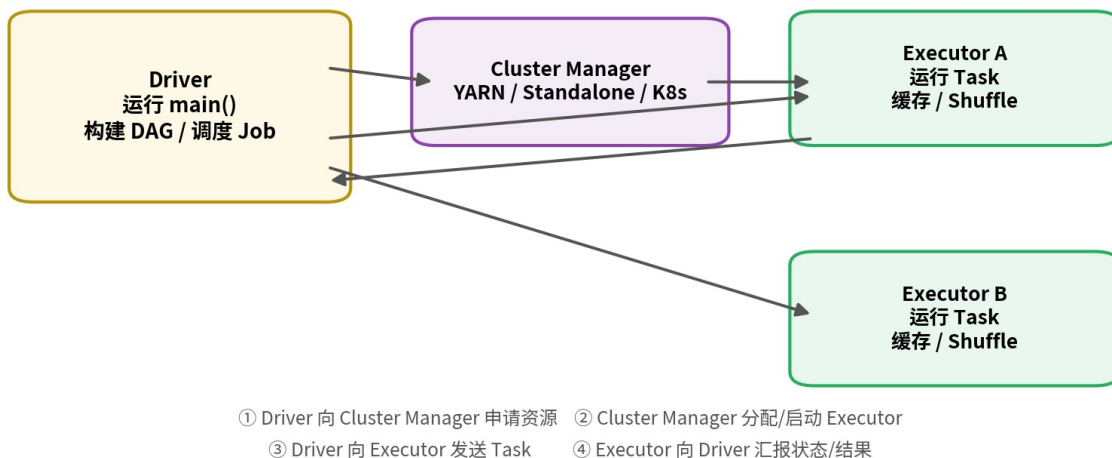
Driver 是运行用户主程序的进程。你的 Python 主逻辑、RDD 依赖关系、Job/Stage 生成、任务调度等核心控制逻辑都从这里发起。

- 运行你的 main 程序（PySpark 中就是脚本主流程）。
- 维护 SparkContext / SparkSession。
- 记录 Transformation 形成的依赖关系。
- Action 出现时创建 Job。
- DAG Scheduler 将 Job 切分为 Stage。
- Task Scheduler 将 Task 分发给 Executor。
- 接收任务状态、Accumulator 更新，以及 collect() 等返回结果。

重要边界：Driver 不应该承担海量数据本身的集中计算。真正的数据分区主要在 Executor 上处理。

五、Cluster Manager 与 Executor：谁给资源，谁真正计算？

Spark Application：Driver、Cluster Manager 与 Executor



一句话：Driver 负责“想清楚怎么干”，Executor 负责“真正把各个分区算掉”。

Spark 本身可以与多种集群资源管理器配合。你当前 Hadoop 学习路线中最自然的是 YARN：Spark Driver 向 YARN 申请资源，集群中启动 Executor，之后由 Executor 长时间运行并不断接收 Task。

组件	主要回答的问题
Cluster Manager	“给这个 Spark Application 多少 CPU / 内存？在哪些节点启动 Executor？”
Executor	“我拿到 Task 以后，真正去处理某个/某些 Partition。”
Driver	“整个应用应该分成哪些 Job/Stage/Task，Task 发给谁？”

一个 Spark Application 通常可以粗略理解成：1 个 Driver + 若干 Executor。Executor 不是 “一个 Task”；一个 Executor 可以连续执行很多 Task。

六、Partition：Spark 并行计算真正落脚的数据单位

RDD 是“分布式数据集”，但真正执行时并不是所有数据一起算，而是被切成多个 Partition。每个 Stage 中，Spark 会针对需要处理的 Partition 生成 Task。

概念	不是	更准确的理解
Partition	不是一台机器	RDD/DataFrame 的一份逻辑数据分片
Task	不是一条数据	某个 Stage 中处理一个 Partition 的一次执行工作
Executor	不是一个 Task	工作进程，可以先后/并发执行多个 Task

记忆法：Partition = 一份数据；Task = 处理这一份数据的工作；Executor = 真正干活的进程。

七、逐行追踪 WordCount：前几行为什么没有马上计算？

这四行主要发生什么？

```
lines = sc.textFile("hdfs:///input/words.txt")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

这些语句会构造新的 RDD 和依赖关系，但通常不会立即把整个 HDFS 文件读完并完成所有计算。这就是上一讲讲过的 Lazy Evaluation。

Driver 此时更像是在画一张“数据加工流程图”：从 lines 到 words，再到 pairs，再到 counts。

为什么要懒执行：因为只有看到完整依赖链，Spark 才能知道哪里存在 Shuffle、怎样切 Stage，以及哪些窄依赖操作可以在同一个 Task 流水线中连续完成。

八、saveAsTextFile()：为什么这一行会让整个系统突然“动起来”？

Action

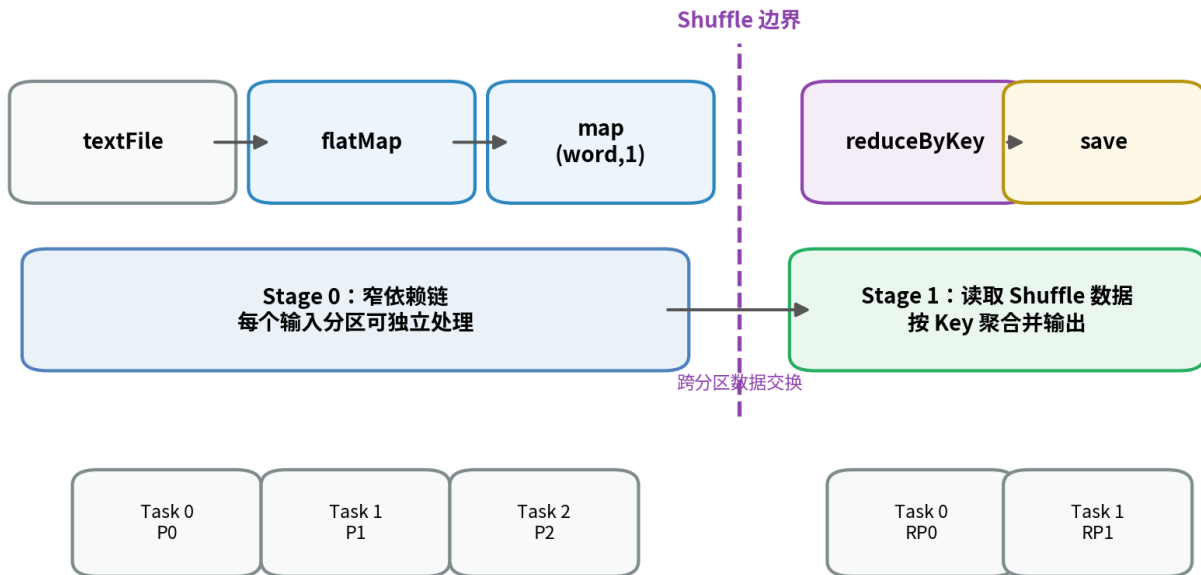
```
counts.saveAsTextFile("hdfs:///output/wordcount")
```

saveAsTextFile() 需要真实输出结果，因此它是 Action。此时 Driver 不能再只记计划，而必须要求集群把前面的所有依赖真正算出来。

1. Driver 创建一个 Job。
2. DAG Scheduler 从最终 RDD 反向查看依赖链。
3. 发现 reduceByKey 是宽依赖，需要跨 Partition Shuffle。
4. 以 Shuffle 为主要边界切分 Stage。
5. 每个 Stage 根据 Partition 数生成一批 Task。
6. Task Scheduler 将 Task 发送给可用 Executor。
7. Executor 读取数据、执行函数、写 Shuffle 或最终输出。

九、为什么 WordCount 通常是两个 Stage?

WordCount：为什么通常会被切成两个 Stage



Stage 内 Task 数通常由该 Stage 的分区数决定；Shuffle 前后分区数可以不同。

textFile -> flatMap -> map 这几步可以让每个输入 Partition 独立完成，不需要知道其他 Partition 里有什么，因此属于典型窄依赖链。

reduceByKey 不一样：同一个单词可能同时出现在 P0、P1、P2 中，必须把相同 Key 的记录重新汇聚。这就需要 Shuffle，于是形成一个重要 Stage 边界。

本质：Stage 不是按“代码行数”切的，而主要按数据依赖边界切。一个 Stage 里可能融合很多个 Transformation。

十、Job、Stage、Task 到底是什么层级关系？

Job、Stage、Task：三层关系



Action -> Job ; Shuffle 等宽依赖通常切 Stage ; 每个 Stage 再按 Partition 生成多个 Task。

层级	由什么产生	数量直觉
Job	通常由一次 Action 触发	一个程序可以有多个 Action，因此可能有多个 Job
Stage	Job 内按 Shuffle/宽依赖等边界切分	一个 Job 常有一个或多个 Stage
Task	Stage 按 Partition 生成并行工作	一个 Stage 通常有很多 Task

因此不要说“一个 Spark 程序就是一个 Job”。如果脚本先 count(), 后 saveAsTextFile(), 通常会触发不止一次 Job；若没有缓存，前面的依赖可能被再次计算。

十一、用一个具体数字理解 Task 数量

假设输入数据当前形成 3 个输入 Partition，并且 reduceByKey 后设置为 2 个分区。可以粗略理解为：

阶段	Partition 数	可能的 Task 数
Stage 0: 读取 + flatMap + map	3	通常 3 个 Task
Shuffle 后的 Stage 1: reduceByKey + save	2	通常 2 个 Task

Task 不是按“有几台机器”决定。即使只有 2 个 Executor，也可以排队执行 100 个 Task；有多少 Task 与数据分区关系更直接，有多少能同时运行则受 Executor 数量和 CPU core 等资源限制。

并行度的两个问题：“总共多少 Task？”和“同一时刻能跑多少 Task？”不是一回事。前者更多来自 Partition，后者更多受 Executor/CPU 资源约束。

十二、reduceByKey 背后发生了什么？

reduceByKey 看起来只是一行 Python，但它是 Spark 中最典型的宽依赖操作之一。假设三个输入分区产生：

Shuffle 前

```
P0: (cat,1) (dog,1)
P1: (cat,1) (bird,1)
P2: (dog,1) (cat,1)
```

Spark 必须让相同 Key 到达同一目标分区，形成类似：

Shuffle 后（示意）

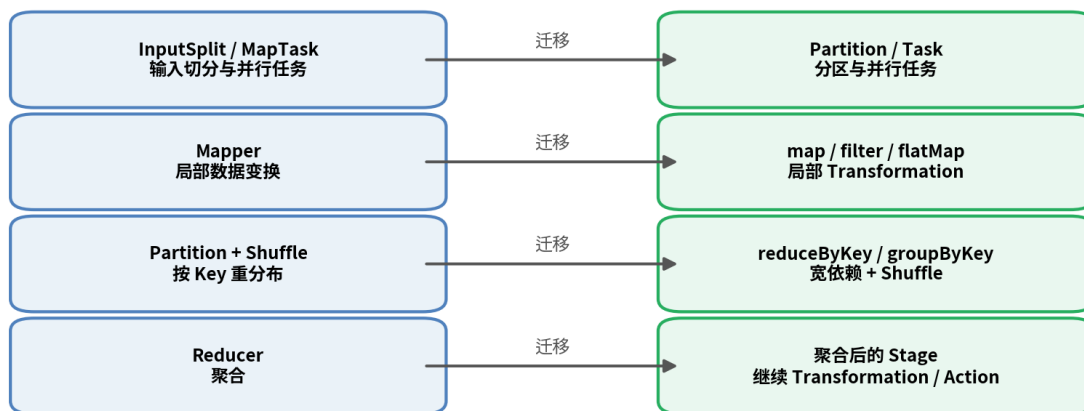
```
目标分区 A: cat -> 1,1,1
目标分区 B: dog -> 1,1   bird -> 1
```

然后在目标分区内进行聚合。这和前面 MapReduce 的 Partition + Shuffle + Reduce 思想高度相似。

迁移关系： MapReduce 教会你的 Shuffle、Partition、数据倾斜并没有失效；Spark 只是用更灵活的 DAG 来组织这些步骤。

十三、Spark 的 map 与 Hadoop Mapper，不要混为一个概念

把 MapReduce 心智模型迁移到 Spark



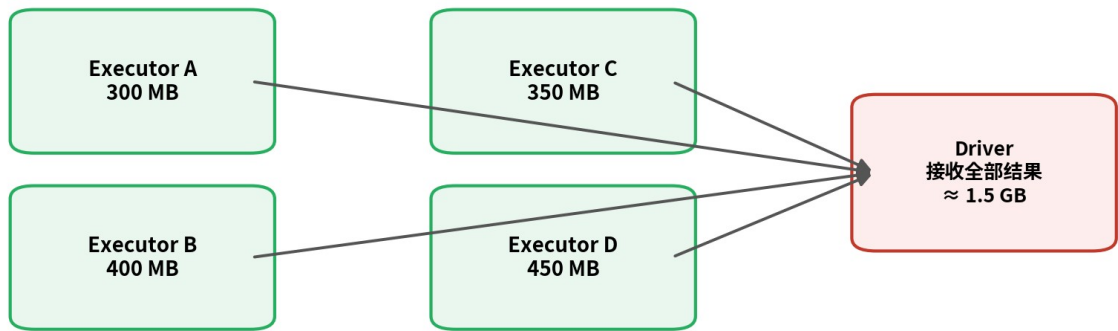
注意： Spark 的 map 不是 Hadoop Mapper 类；Spark 的 reduceByKey 也不是 Hadoop Reducer 类。思想相通，编程模型不同。

两者名字相似，但抽象层级不同。Hadoop Mapper 是 MapReduce 框架规定的一个阶段/类；Spark map() 只是一个普通窄依赖 Transformation，可以在一个 Stage 中与 filter、flatMap 等很多操作流水化执行。

同样，Spark `reduceByKey()` 不是“调用 Hadoop Reducer”。它是 Spark 自己的 Key 聚合操作，会建立宽依赖并触发 Spark Shuffle。

十四、`collect()` 为什么危险？

为什么 `collect()` 是初学者最容易踩的坑之一



`collect()` 的语义是“把所有分区结果拉回 Driver”。小结果没问题，大结果可能直接把 Driver 内存打爆。

`collect()` 也是 Action。它要求所有 Executor 把结果发送回 Driver，并在 Driver 进程中形成完整结果集合。如果结果只有几百条，`collect()` 很方便；如果结果有几 GB、几十 GB，Driver 很可能内存溢出。分布式系统最忌讳“前面分布式算得好好的，最后突然全部搬回一台机器”。

需求	更合适的选择
调试，只看前几条	<code>take(10) / show()</code> (DataFrame)
统计数量	<code>count()</code>
保存海量结果	写 HDFS / Parquet 等分布式存储
确实是很小的最终结果	可以 <code>collect()</code>

十五、Driver 和 Executor 谁运行你的 Python 函数？

这是 PySpark 初学者的高频疑问。你在 Driver 上定义 `lambda`/函数，但 Spark 会把需要用于 Task 的函数和必要依赖序列化后发送到 Executor，Executor 上的 Python worker 对对应 Partition 数据执行它。

容易错的地方：不要在 `map()` 内假设可以随意修改 Driver 上的普通 Python 变量并让所有节点共享结果。Executor 是分布式进程，普通变量不是“全局共享内存”。

如果确实需要“小型只读数据”分发到各 Executor，Spark 有 Broadcast Variable；如果要做跨 Task 的受控累积统计，有 Accumulator 等机制。它们后面会单独讲。

十六、Local 模式和真正集群模式有什么差别？

模式	Driver / Executor 所在位置	适合
local[*]	主要在本机进程/线程环境模拟并行	学习、开发、快速调试
YARN / Standalone / Kubernetes	资源分布在集群节点	真正大规模数据处理

代码可以非常相似，但运行环境不同。Local 模式帮助你理解 API，不等于真实集群里不存在网络、Shuffle、资源调度等问题。

一个重要习惯：学习 Spark 时可以先 local 模式验证逻辑，再放到 YARN/HDFS 环境验证分布式行为。

十七、如果以后在 Hadoop + YARN 环境上跑 Spark，控制流会怎样？

在 YARN 上执行 spark-submit 时，可以把整体流程理解为：

1. spark-submit 提交 Spark Application。
2. YARN 为应用分配资源。
3. Driver 启动（具体位置与 deploy mode 有关）。
4. Driver 申请并管理 Executor 资源。
5. Executor 启动后向 Driver 注册。
6. Action 触发 Job，Driver 产生 Stage / Task。
7. Task 被发送到 Executor，读取 HDFS 分区并执行。
8. 出现 Shuffle 时 Executor 之间交换中间数据。
9. 最终结果写 HDFS，Application 结束并释放资源。

这里最重要的不是死记 client/cluster 模式细节，而是建立“YARN 管资源，Spark Driver 管 Spark 计算，Executor 干活”的层次。

十八、上机观察：如何亲眼看到 Job / Stage / Task

前提：下面命令用于已经安装并配置 Spark 的环境。如果当前虚拟机尚未安装 Spark，可以先把原理学完，等课程进入 Spark 环境配置后再执行。

1) 先确认 Hadoop/YARN 可用（YARN 场景）

```
# 如果使用 HDFS / YARN，先启动并检查 Hadoop 服务
start-dfs.sh
start-yarn.sh

jps
hdfs dfsadmin -report
yarn node -list
```

2) 确认 Spark 命令可用

```
# Spark 安装好以后，可先检查
spark-submit --version
pyspark --version
```

3) 提交应用（示意）

```
# 本地模式运行脚本（学习阶段常用）
spark-submit --master local[*] wordcount.py

# 后续在配置好的 YARN 环境中可使用类似：
spark-submit --master yarn wordcount.py
```

应用运行期间，Spark Web UI 通常可以观察 Jobs、Stages、Tasks、Executors、Storage、SQL 等页面。真正理解 Spark 的最快方式之一，就是把“代码里的一个 Action”对应到 UI 里出现的 Job，再进入 Stage 看 Task 数量。

十九、学会读 Spark UI：不要只看“成功/失败”

页面/信息	应该问什么
Jobs	哪个 Action 触发了这个 Job?
Stages	为什么会切成这些 Stage? 哪个位置发生 Shuffle?
Tasks	Task 数量为什么是这个数? 是否出现明显长尾?
Executors	各 Executor 的 Task、内存、Shuffle 读写是否均衡?
Storage	哪些 RDD/DataFrame 被缓存? 占了多少内存?

这和前面 MapReduce 中学习 Counters/YARN 日志是同一个方法论：不要只看最终结果，要观察系统留下的执行证据。

二十、14 个初学者最常见的疑问

1. SparkSession 是 Executor 吗?

答：不是。SparkSession 是 Driver 侧的高层入口对象。

2. SparkContext 是一台服务器吗?

答：不是。它是 Driver 中代表 Spark 应用执行上下文的对象。

3. 一个 Executor 只运行一个 Task 吗?

答：不是。Executor 生命周期内可以执行许多 Task，并可并发运行多个 Task。

4. 一个 Partition 就对应一台机器吗?

答：不是。Partition 是逻辑数据分片，Task 会被调度到某个 Executor。

5. 一个 Task 会处理整个 RDD 吗?

答：通常不会。Task 处理某个 Stage 中的一个 Partition。

6. 一个 Action 一定只有一个 Stage 吗？

答：不一定。若依赖链中有 Shuffle，通常会有多个 Stage。

7. 一个程序一定只有一个 Job 吗？

答：不一定。多个 Action 可以触发多个 Job。

8. map() 会立即执行吗？

答：通常不会，它是 Transformation，先记录依赖。

9. reduceByKey() 会立即执行吗？

答：它是 Transformation，本身仍是惰性的，但建立了 Shuffle 宽依赖。

10. collect() 为什么也是 Action？

答：因为它必须真正得到所有结果并拉回 Driver。

11. Spark 没有 Mapper/Reducer，是不是就没有 Shuffle？

答：错误。reduceByKey、groupByKey、join 等操作仍可能发生 Shuffle。

12. Executor 挂了数据就全部丢了吗？

答：缓存数据可能丢失，但 Spark 可依赖 lineage 等机制重算必要分区；Shuffle/任务也有相应恢复机制。

13. Task 越多越好吗？

答：不是。过少并行度不足，过多又会增加调度与小任务开销。

14. Spark 一定要把所有数据放内存吗？

答：不是。数据可能来自磁盘，也可能落盘/溢写；“内存计算”指可以高效复用内存中的中间数据。

二十一、迁移练习：你来推演一个 Spark Job

假设 HDFS 中有 1 TB 订单数据，当前输入被划分为 100 个 Partition。程序逻辑：过滤 PAID 订单 -> 映射成 (city, amount) -> reduceByKey 求城市销售额 -> 保存到 HDFS；reduceByKey 的目标分区数设为 20。

问题	先自己回答
1	filter 和 map 是否需要 Shuffle？为什么？
2	reduceByKey 为什么形成宽依赖？
3	这个 Job 大致可以分成几个 Stage？
4	Shuffle 前 Stage 大致会有多少 Task？
5	Shuffle 后 Stage 大致会有多少 Task？
6	如果 Executor 只有 5 个、每个 4 cores，100 个 Task 会怎样执行？

参考思路：filter/map 是窄依赖；reduceByKey 要让相同 city 跨分区汇聚，因此需要 Shuffle。典型情况下可形成 Shuffle 前后两个 Stage；前一 Stage 约 100 个 Task，后一 Stage 约 20 个 Task。若同时可用约 20 个 task slot/cores，则 100 个 Task 会分多批调度执行。

二十二、本讲最后形成一条完整心智链

从代码到物理执行

```

PySpark 代码
↓
SparkSession / SparkContext (Driver 侧入口)
↓
Transformation: 先建立依赖关系
↓
Action: 触发 Job
↓
DAG Scheduler: 按宽依赖 / Shuffle 切 Stage
↓
每个 Stage 按 Partition 生成 Task
↓
Task Scheduler 将 Task 发给 Executor
↓
Executor 读取分区、执行函数
↓
必要时 Shuffle
↓
最终结果写出 / 返回 Driver

```

本讲最重要的结论：Spark 的“魔法”不是 Python 代码自动变快，而是把数据依赖、分区、Shuffle 和集群资源统一组织成 Job -> Stage -> Task 的执行体系。你理解了这条链，后面学习 DataFrame、Spark SQL、缓存和性能优化就不再是背 API。

二十三、下一讲预告

下一讲建议继续深入 RDD：重点讲 Narrow Dependency / Wide Dependency、Lineage、cache/persist、为什么缓存能加速迭代计算、缓存什么时候反而浪费内存，以及 RDD 分区丢失后 Spark 如何通过 Lineage 重算。之后再从 RDD 过渡到现在实际工程中更常用的 DataFrame / Spark SQL。